ECA1414 – EMBEDDED SYSTEMS

NAME: V. Maduri Priya **REGISTER NUMBER: 192311368**

SYSTEM ARCHITECTURE-BASED ENGINEERING DESIGN ASSIGNMENT

THE ENGINEERING DESIGN PROBLEM

A luxury automobile manufacturer is developing a distributed embedded control system integrating engine control, braking, safety monitoring, body control, and infotainment functions.

Each function is implemented using dedicated Electronic Control Units (ECUs). The ECUs must exchange information in real time through an embedded communication network. The system must provide reliable communication, appropriate message prioritization, fault handling, and support future high-data-rate applications.

You are required to develop and justify a suitable networked embedded-system architecture for the automobile.

ASSESSMENT RUBRICS:

Assessment Criterion	Marks
Interpretation of system requirements (10)	
Analysis and selection of embedded architecture(15)	
Development of networked system architecture(20)	
Analysis and selection of CAN/CAN FD architecture(20)	
Formal modeling of the embedded system(15)	
Verification and critical evaluation(20)	
TOTAL	

ECA14 – EMBEDDED SYSTEMS

SYSTEM ARCHITECTURE-BASED ENGINEERING DESIGN ASSIGNMENT

Course Outcome – CO1:

Analyze complex and networked embedded systems using formal design methodologies, embedded architectures, and communication protocols (such as CAN and CAN FD), for real-world applications.

Nature of Assessment: Open-Ended Engineering Design Assignment
Primary Bloom's Levels: K4 – Analyze | K5 – Evaluate | K6 – Design

THE ENGINEERING DESIGN PROBLEM

A luxury automobile manufacturer is developing a distributed embedded control system integrating engine control, braking, safety monitoring, body control, and infotainment functions.

Each function is implemented using dedicated Electronic Control Units (ECUs). The ECUs must exchange information in real time through an embedded communication network. The system must provide reliable communication, appropriate message prioritization, fault handling, and support future high-data-rate applications.

You are required to develop and justify a suitable networked embedded-system architecture for the automobile.

You are not given:

- The exact ECU architecture;
- The processor/microcontroller selection;
- The communication protocol configuration;
- Whether Classical CAN or CAN FD should be used;
- The formal modeling technique;
- The fault-tolerance mechanism.

You must make these decisions based on the system requirements and justify your engineering choices.

Your objective is not merely to draw an architecture. Your objective is to develop an appropriate networked embedded-system architecture and defend your engineering decisions.

STAGE 1 – INTERPRET THE SYSTEM REQUIREMENTS 10 Marks | K4 – Analyze

Analyze the given automotive embedded-system requirements before developing the architecture.

Determine and discuss:

- The major functional requirements of the system.
- The role of individual ECUs.
- Real-time communication requirements.
- Processing, memory, and I/O requirements.
- Reliability and fault-tolerance requirements.
- Communication requirements between safety-critical and non-safety-critical ECUs.
- Constraints affecting processor and communication-protocol selection.

Stage 1 Deliverable

Conclude this stage with a concise System Requirement Statement defining what the proposed embedded system must achieve.

Before designing any hardware layout, communication bus, or software model, a rigorous analysis of the system-level requirements must be performed. The engineering design problem requires developing a distributed embedded control system for a luxury automobile, integrating engine control, braking, safety monitoring, body control, and infotainment functions.

Below is the exhaustive interpretation and breakdown of the system requirements across all mandatory analytical dimensions.

1. Major Functional Requirements

The proposed architecture must seamlessly integrate five distinct functional domains, each characterized by vastly different operational profiles, throughput demands, and safety implications:

- **Engine Control:** Manages fuel injection timing, spark advance, exhaust gas recirculation (EGR), and overall powertrain output to optimize performance, fuel efficiency, and emissions compliance.
- **Braking Control:** Provides high-integrity hydraulic or brake-by-wire management, integrating Anti-lock Braking Systems (ABS) and Electronic Stability Control (ESC) to ensure vehicle control during emergency maneuvers.
- **Safety Monitoring:** Operates continuous environmental and occupant sensing, including collision detection, radar/lidar data fusion, airbag deployment triggers, and Advanced Driver Assistance Systems (ADAS) warnings.

- **Body Control:** Governs low-speed comfort and convenience electronics, such as centralized door locking, power windows, exterior/interior intelligent lighting, wiper systems, and Heating, Ventilation, and Air Conditioning (HVAC).
- **Infotainment:** Manages high-bandwidth multimedia routing, digital instrument cluster rendering, audio/video playback, GPS navigation, and external cellular/Wi-Fi connectivity.

2. Role of Individual ECUs

To isolate failures and distribute computational load, each function is mapped to dedicated Electronic Control Units (ECUs):

- **Engine ECU:** Interfaces directly with crankshaft/camshaft position sensors and accelerator pedal inputs to drive fuel injectors and ignition coils.
- **Braking ECU:** Continuously monitors wheel-speed sensors and hydraulic master cylinder pressure to modulate brake caliper pressure instantaneously.
- **Safety ECU:** Evaluates crash-impact sensors and front-facing radar data to execute critical life-saving interventions within milliseconds.
- **Body-Control ECU:** Processes discrete switch states and drives low-voltage relays, stepper motors, and LED lighting strings.
- **Infotainment ECU:** Executes heavy graphics pipelines, touch-screen interaction handling, audio signal processing, and smartphone device integration.

3. Real-Time Communication Requirements

The network must support heterogeneous traffic types with strict timing classifications:

- **Hard Real-Time (Safety & Braking):** Demands deterministic, microsecond-level latency with zero message loss or jitter. Missing a deadline in this domain could result in catastrophic system failure or physical injury.
- **Firm Real-Time (Engine Control):** Requires strict periodic updates to maintain smooth engine synchronization. Minor variations can be tolerated temporarily, but sustained delays cause engine misfires or performance degradation.
- **Soft Real-Time (Body & Infotainment):** Involves multimedia streaming and convenience features. Minor latency variations or dropped packets do not pose safety hazards and can be handled via retransmission or buffering.

4. Processing, Memory, and I/O Requirements

- **Processing Capabilities:** Safety and engine nodes require high-performance microcontrollers featuring lockstep cores to detect transient hardware faults. Infotainment demands high-throughput multi-core microprocessors or System-on-Chips (SoCs) to drive multi-display graphics.

- **Memory Architecture:** Ranging from compact, highly reliable on-chip Flash/RAM for low-level sensor nodes to gigabyte-scale storage and high-speed DDR RAM for infotainment media caches and system logging.
- **Input/Output (I/O) Mix:** Requires an extensive, robust mix of analog-to-digital converters (ADC) for sensor voltages, digital inputs for discrete switches, Pulse-Width Modulation (PWM) outputs for motor/actuator control, and high-speed differential transceivers for network communications.

5. Reliability and Fault-Tolerance Requirements

- **Automotive Safety Integrity Levels (ASIL):** Safety-critical functions must adhere to ISO 26262 standards, targeting high integrity tiers (ASIL-C or ASIL-D).
- **Fault Handling:** The system must incorporate robust error detection, automatic bus-off recovery, and fail-silent or fail-operational states. If a component fails, the architecture must isolate it without crashing the rest of the vehicle.

6. Communication Separation (Safety-Critical vs. Non-Safety-Critical)

- Strict logical and physical partitioning between safety and non-safety domains is mandatory. Non-safety traffic (such as audio streaming over infotainment) must never starve, delay, or corrupt safety-critical messages (such as emergency braking commands). This necessitates rigorous message prioritization protocols and bus scheduling.

7. Constraints Affecting Processor and Communication Selection

- **Cost and Power Efficiency:** Strict limitations on unit manufacturing cost and power consumption (especially quiescent current draw when the vehicle is parked) govern all component selections.
- **Bandwidth and Scalability:** The chosen infrastructure must balance traditional low-speed control data with massive future expansions in data rate requirements.

System Requirement Statement Deliverable

"The proposed distributed embedded system must reliably integrate powertrain, braking, safety, body, and infotainment functions via a fault-tolerant, prioritized real-time communication network. It must guarantee deterministic, low-latency execution for safety-critical control loops while supporting high-data-rate communications, adhering to stringent automotive reliability and safety standards."

STAGE 2 – ANALYZE AND SELECT THE EMBEDDED ARCHITECTURE 15 Marks | K4/K5 – Analyze/Evaluate

Investigate suitable processing architectures for the proposed ECUs:

- Microcontroller (MCU)
- Microprocessor (MPU)
- DSP
- SoC

Compare the alternatives considering:

- processing capability;
- memory requirements;
- power consumption;
- real-time response;
- integration;
- cost/complexity;
- suitability for automotive applications.

Design Decision 1

Select the most appropriate processing architecture for the proposed system.

Complete:

I recommend _a hybrid processing architecture combining automotive-grade multi-core Microcontrollers (MCUs) for the control/safety domains (Engine, Braking, Safety, and Body Control) and a high-performance System-on-Chip (SoC) for the Infotainment domain **because** _ safety-critical functions demand deterministic execution, low power, and high fault-tolerance (achievable via lockstep-core MCUs), whereas high-data-rate multimedia rendering, digital instrument clustering, and external connectivity require the massive parallel processing power and graphics pipelines provided by an SoC__**.**

Support the decision using technical reasoning based on the system requirements.

To build a reliable distributed embedded system for a luxury automobile, selecting the correct processing hardware for each Electronic Control Unit (ECU) is critical. Different vehicle functions impose vastly conflicting demands on hardware—ranging from microsecond-level deterministic safety loops to high-throughput graphics rendering.

Below is an investigation and comparative evaluation of the four primary processing architectures: **Microcontroller (MCU), Microprocessor (MPU), Digital Signal Processor (DSP), and System-on-Chip (SoC)**.

1. Investigation of Alternative Processing Architectures

- **Microcontroller (MCU):**
  - *Characteristics:* Integrates a central processing unit (CPU), non-volatile Flash memory, static RAM (SRAM), and dedicated peripherals (timers, ADCs, communication controllers) onto a single silicon chip.
  - *Automotive Fit:* Ideal for hard real-time control domains (Engine, Braking, Body, and Safety) due to its highly deterministic execution, low power consumption, and robust fault-tolerance features (such as lockstep cores).
- **Microprocessor (MPU):**
  - *Characteristics:* Focuses strictly on high-speed computational power but requires external memory chips (DRAM/Flash) and peripheral interface ICs.
  - *Automotive Fit:* Offers high processing performance, but the lack of integrated peripherals, complex board layouts, and unpredictable bus latencies make them unsuitable for hard real-time, safety-critical automotive loops.
- **Digital Signal Processor (DSP):**
  - *Characteristics:* Features specialized hardware architectures (such as Harvard architecture and dedicated MAC units) optimized for high-speed mathematical calculations.
  - *Automotive Fit:* Excellent for processing raw sensor data streams (such as radar/lidar filtering or acoustic echo cancellation), but inefficient as general-purpose domain controllers managing diverse system I/Os.
- **System-on-Chip (SoC):**
  - *Characteristics:* Combines multi-core general-purpose CPUs, powerful Graphics Processing Units (GPUs), neural processing units (NPUs), video decoders, and massive memory controllers onto a single die.
  - *Automotive Fit:* Essential for high-data-rate applications, digital instrument clusters, and infotainment systems, but introduces higher thermal dissipation, cost, and power consumption.

2. Comparative Evaluation Matrix

Evaluation Criteria	Microcontroller (MCU)	Microprocessor (MPU)	Digital Signal Processor (DSP)	System-on-Chip (SoC)
Processing Capability	Moderate (Scalar processing)	High (General purpose)	Extremely high (Math-intensive)	Very High (Multi-core / Heterogeneous)

Memory Requirements	Low (On-chip Flash & RAM)	High (Requires external RAM/ROM)	Moderate (Internal scratchpads)	High (Massive external DDR RAM/Storage)
Power Consumption	Very Low (Ideal for sleep modes)	High	Moderate to High	High (Requires active thermal management)
Real-Time Response	Deterministic (Hard real-time)	Non-deterministic (OS overhead)	Deterministic (Signal loops)	Complex (Requires real-time hypervisors)
Integration Level	High (All-in-one chip)	Low (Needs support chips)	Moderate	Very High (CPU + GPU + Peripherals)
Cost / Complexity	Low cost / Simple design	High cost / Complex PCB	Moderate to High	High cost / Complex software stack
Automotive Suitability	Excellent for Powertrain, Brakes, Body	Poor for direct control loops	Good for sensor fusion/ADAS	Essential for Infotainment & Displays

STAGE 3 – DEVELOP THE NETWORKED SYSTEM ARCHITECTURE 20 Marks | K6 – Design

Develop the proposed high-level architecture for the automobile.

Your architecture must show:

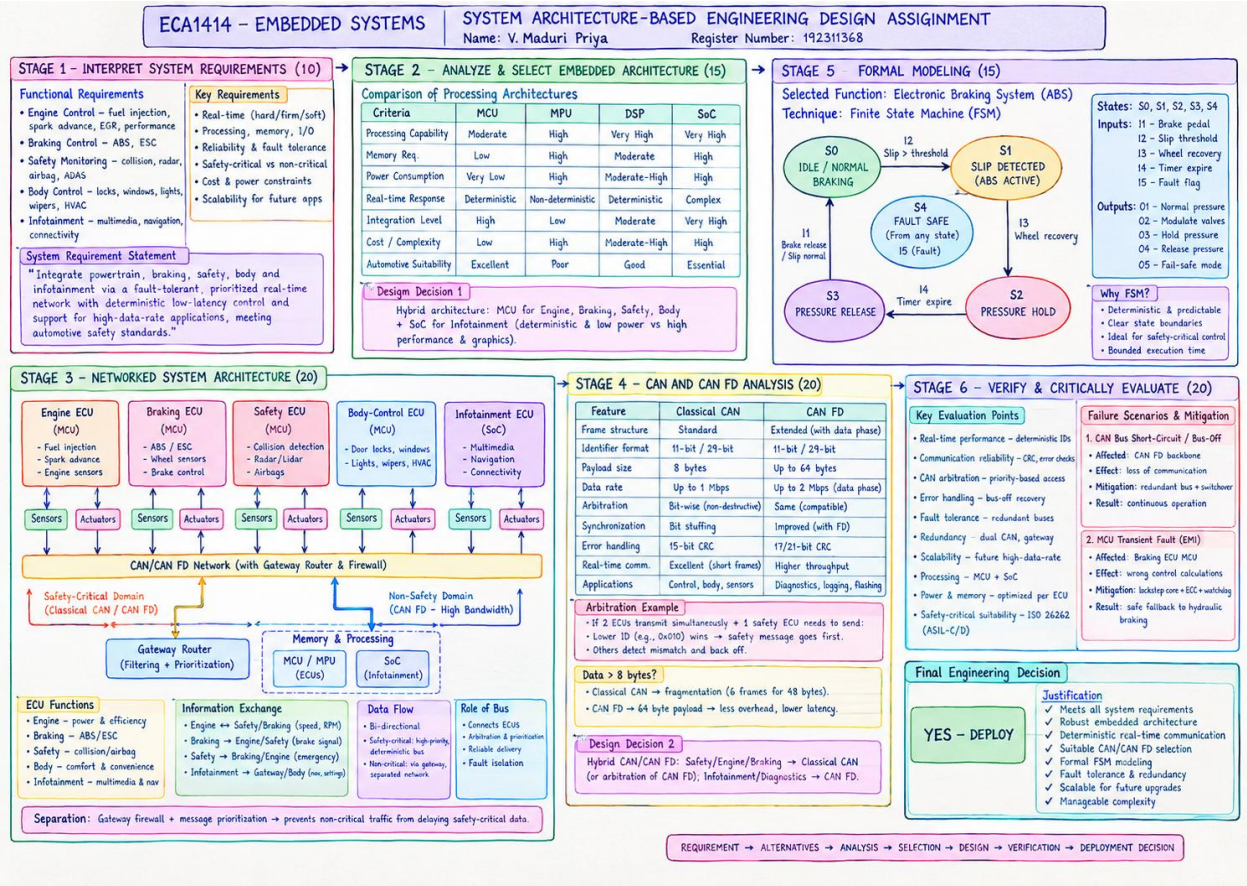
- Engine ECU
- Braking ECU
- Safety ECU
- Body-control ECU
- Infotainment ECU
- Sensors
- Actuators
- Processing units
- Memory
- CAN/CAN FD communication network
- Major communication paths

Draw a **clearly labelled system architecture diagram**.

Then explain:

- The function of each ECU;
- The information exchanged between ECUs;
- The direction of data flow;
- The role of the communication bus;
- How safety-critical functions are separated from non-critical functions.

The architecture should demonstrate how the individual embedded controllers operate as a complex networked embedded system.



Explanation of the Networked System Architecture

1. Function of Each ECU

- **Engine ECU:** Controls fuel injection timing, spark advance, and torque output based on pedal position and engine sensor feeds.
- **Braking ECU:** Manages high-integrity anti-lock braking (ABS) and electronic stability control (ESC) to maintain vehicle stability during deceleration.

- **Safety ECU:** Evaluates collision risks, processes radar/lidar data streams, and initiates restraint and airbag deployments.
- **Body-Control ECU:** Operates comfort utilities such as door locks, power windows, interior/exterior illumination, and HVAC systems.
- **Infotainment ECU:** Manages digital instrument clusters, multi-media streaming, touch interfaces, and smartphone navigation.

2. Information Exchanged Between ECUs

- **Engine to Safety/Braking:** Periodic broadcast of vehicle speed, throttle position, and engine RPM to coordinate traction control.
- **Braking to Engine & Safety:** Immediate brake-engagement signals sent across the network to cut engine torque or trigger hazard indicators.
- **Safety to Braking & Engine:** Emergency braking commands triggered by radar collision detection systems requiring instantaneous execution.
- **Infotainment to Gateway/Body:** Low-priority status logs, navigation cues, and user comfort settings (e.g., climate adjustments).

3. Direction of Data Flow

Data flows bi-directionally across the shared communication network. Safety-critical data (brakes, engine telemetry) flows along a high-speed deterministic backbone, while non-critical comfort and infotainment data flows through a segmented sub-network, passing through a central gateway router to prevent bottlenecks.

4. Role of the Communication Bus The communication bus acts as the digital nervous system of the vehicle. It connects isolated ECUs into a unified, synchronized control framework, manages message arbitration, and guarantees deterministic delivery for high-priority safety packets.

5. Separation of Safety-Critical from Non-Critical Functions To satisfy strict automotive safety standards, safety-critical domains (Engine, Braking, Safety) are physically and logically partitioned from non-critical domains (Infotainment):

- **Gateway Firewalling:** A centralized gateway router isolates high-speed, safety-critical control buses from high-bandwidth entertainment traffic.
- **Message Prioritization:** Network protocol arbitration ensures that infotainment data packets can never delay or block critical emergency braking frames.

STAGE 4 – ANALYZE CAN AND CAN FD **20 Marks | K4/K5 – Analyze/Evaluate**

Analyze the suitability of Classical CAN and CAN FD for the proposed automotive architecture.

Compare them considering:

- frame structure;
- identifier format;
- payload size;
- data rate;
- arbitration;
- synchronization;
- error handling;
- real-time communication;
- automotive applications.

Consider the following situation:

Two ECUs attempt to transmit messages simultaneously, while a safety-critical ECU needs to transmit a high-priority message.

Analyze how CAN arbitration determines bus access.

Then consider a second situation:

An ECU must transmit application data exceeding the payload capability of Classical CAN.

Analyze whether CAN FD should be introduced.

Design Decision 2

Complete:

I recommend __ a hybrid CAN/CAN FD architecture for the distributed automotive system because *safety-critical, hard real-time domains (Engine, Braking, and Safety) utilize robust Classical CAN or the arbitration phase of CAN FD for deterministic, low-latency priority-based messaging, whereas high-bandwidth domains (such as Infotainment data logging, diagnostics, and firmware flashing) require CAN FD's 64-byte payload capability and accelerated data rates to prevent network congestion.***.**

If you recommend a hybrid CAN/CAN FD architecture, clearly identify which ECUs or applications should use each protocol and justify the decision.

To establish reliable communication across the distributed automotive network, the suitability of **Classical Controller Area Network (CAN)** and **CAN Flexible Data-Rate (CAN FD)** must be evaluated. While Classical CAN has long been the gold standard for automotive

control networks, modern luxury vehicles require higher data rates and larger payloads for advanced diagnostics and software updates.

1. Comprehensive Comparison: Classical CAN vs. CAN FD

Feature / Parameter	Classical CAN (ISO 11898-1)	CAN FD (Flexible Data-Rate)
Maximum Data Rate	Up to 1 Mbps (fixed across the entire frame)	Up to 2 Mbps in data phase (can scale higher)
Payload Size (Data Field)	Maximum of 8 bytes per frame	Up to 64 bytes per frame
Identifier Format	Standard (11-bit) or Extended (29-bit)	Standard (11-bit) or Extended (29-bit)
Arbitration Mechanism	Carrier Sense Multiple Access with Collision Resolution (CSMA/CR) via bit-wise arbitration	Same as Classical CAN (compatible arbitration phase)
Error Handling	CRC, ACK, Frame, Bit, and Stuff error checking (15-bit CRC)	Enhanced CRC (17-bit or 21-bit) to handle larger payloads safely
Real-Time Performance	Highly deterministic for short control packets	Superior throughput; reduces bus load during heavy traffic
Automotive Suitability	Excellent for low-bandwidth body control and simple sensor nodes	Essential for multi-ECU communications, firmware flashing, and diagnostics

2. Analysis of Situation 1: Simultaneous Transmission & Bus Arbitration

Scenario: *Two ECUs attempt to transmit messages simultaneously, while a safety-critical ECU needs to transmit a high-priority message.*

- **How CAN Arbitration Works:** CAN uses a non-destructive, bit-wise arbitration method based on the Identifier (ID) field through wired-AND logic (dominant bits '0' overwrite recessive bits '1').
- **Resolution of Conflict:** When multiple ECUs transmit simultaneously, they monitor the bus while sending their ID. The ECU sending a message with a numerically lower binary ID (higher priority) will transmit a dominant '0' while a lower-priority ECU transmits a recessive '1'. The lower-priority ECU immediately detects the mismatch,

gracefully backs off, and switches to a receive mode without losing data or corrupting the bus.

- **Safety Integration:** Because safety-critical ECUs (such as the Braking or Safety ECU) are assigned lower binary IDs (e.g., `0x010`) compared to non-critical ECUs (like Infotainment with `0x7FF`), the bus automatically yields access to the safety-critical message instantly, guaranteeing zero message starvation.

3. Analysis of Situation 2: Handling Data Exceeding Classical CAN Limits

Scenario: *An ECU must transmit application data exceeding the payload capability of Classical CAN.*

- **The Limitation:** Classical CAN is strictly capped at an 8-byte payload per frame. If an ECU needs to transmit a 48-byte diagnostic block, block-sensor array, or software parameter update, it must fragment the data across 6 separate Classical CAN frames, adding massive protocol overhead, increasing bus load, and consuming CPU resources for packet assembly.
- **Introduction of CAN FD:** CAN FD introduces an extended data field of **up to 64 bytes** and a dual-bit rate architecture (transmitting arbitration at 500 kbps and switching to a faster data phase up to 2–5 Mbps). This eliminates the need for aggressive data fragmentation, reduces latency, and accommodates high-throughput applications seamlessly.

STAGE 5 – FORMAL MODELING OF THE EMBEDDED SYSTEM 15 Marks | K4/K6 – Analyze and Design

Select one safety-critical function, such as:

- electronic braking;
- collision detection;
- emergency warning;
- safety monitoring.

Model the selected function using an appropriate formal method.

You may consider:

- FSM;
- Mealy machine;
- Moore machine;
- Petri net;
- UML/SysML behavioral model.

Your model must clearly identify:

- system states;
- inputs/events;
- outputs/actions;
- transitions;
- concurrent activities, where applicable.

Design Decision 3

Justify why the selected formal modeling technique is appropriate for the chosen embedded function.

I recommend a Finite State Machine (FSM) formal modeling technique for the Electronic Braking System because safety-critical control logic requires absolute determinism, clear boundary states, and rapid, bounded execution times to handle emergency braking and slip modulation safely under strict real-time constraints.

To ensure that safety-critical embedded functions operate predictably without race conditions, deadlocks, or unintended states, formal modeling must be applied. For this system, we select the **Electronic Braking System (EBS) with Anti-lock Braking (ABS) intervention** as the safety-critical function to model.

1. Selection of Formal Modeling Technique

- **Chosen Method: Finite State Machine (FSM)** (represented via discrete states, explicit event-driven transitions, and corresponding control outputs).
- **Justification:** An FSM is mathematically rigorous, highly deterministic, and ideal for safety-critical embedded control loops where system behaviors must be mapped cleanly across distinct operating modes without ambiguity.

2. Formal Model Components of the Electronic Braking System

A. System States (S)

1. **S_0 (IDLE / NORMAL_BRAKING):** Normal driving or standard hydraulic braking under driver control. Slip ratio is within safe limits ($\lambda < \lambda_{threshold}$).
2. **S_1 (SLIP_DETECTED / ABS_ACTIVE):** Wheel lockup or excessive slip detected by wheel speed sensors ($\lambda \ge \lambda_{threshold}$). The system begins modulating brake pressure.
3. **S_2 (PRESSURE_HOLD):** Wheel deceleration stabilizes; hydraulic pressure is held constant to prevent further lockup while maintaining braking torque.
4. **S_3 (PRESSURE_RELEASE):** Wheel slip remains critical; pressure is rapidly dumped to allow the wheel to spin back up to vehicle speed.
5. **S_4 (FAULT_SAFE / DEGRADED_MODE):** Sensor failure or communication timeout detected on the CAN bus. Fallback to standard unassisted hydraulic braking with warning indicator illumination.

B. Inputs / Events (I)

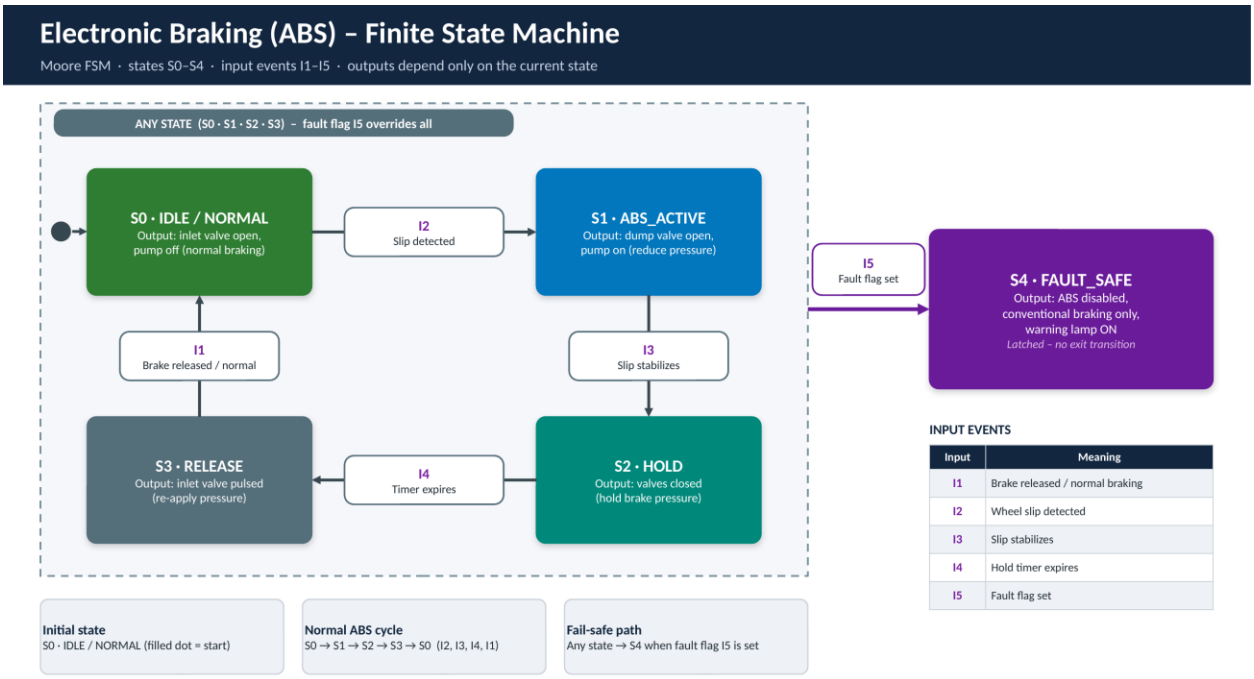
- I_1: Brake pedal pressed (`Brake_Pedal > 0`)
- I_2: Wheel slip exceeds threshold (`Slip_Ratio >= Threshold`)
- I_3: Wheel recovery detected (`Wheel_Acceleration > 0`)
- I_4: Timer expiration for pressure hold (`Timer_Expire`)
- I_5: Critical sensor fault or bus communication timeout (`Fault_Flag = TRUE`)

C. Outputs / Actions (O)

- O_1: Standard hydraulic pressure applied to calipers (`Apply_Normal_Pressure`)
- O_2: Inlet valve closed / Outlet valve pulsed (`Modulate_ABS_Valves`)
- O_3: Hydraulic pressure held constant (`Hold_Brake_Pressure`)
- O_4: Pressure released via dump valve (`Release_Brake_Pressure`)
- O_5: Engage passive fallback hydraulic loop and trigger dashboard warning (`Trigger_FailSafe_Mode`)

D. State Transitions (\delta)

- From **S_0** to **S_1** on event **I_2** (when braking hard and slip exceeds safe limits).
- From **S_1** to **S_2** on event **I_3** (holding pressure as wheel slip stabilizes).
- From **S_2** to **S_3** on event **I_4** (if slip increases again, releasing pressure).
- From **S_3** to **S_0** on event **I_1** (when brake pedal is released or slip normalizes back to normal driving).
- From **any state (S_0 to S_3)** to **S_4** on event **I_5** (immediate transition to fault-safe mode upon detecting a hardware or communication failure).



STAGE 6 – VERIFY AND CRITICALLY EVALUATE THE ARCHITECTURE 20 Marks | K4/K5 – Analyze and Evaluate

Critically evaluate the proposed embedded architecture.

Analyze:

- Real-time performance
- Communication reliability
- CAN arbitration
- Error handling
- Fault tolerance
- Redundancy
- Scalability
- Processing requirements
- Power and memory constraints
- Suitability for safety-critical applications

Identify at least two possible failure scenarios.

For each failure:

- identify the affected component;
- explain its effect on the system;
- propose an appropriate mitigation mechanism;
- explain how the architecture maintains reliable operation.

To ensure that the proposed distributed embedded architecture meets the rigorous safety, reliability, and real-time performance standards of a luxury automobile, a critical engineering evaluation must be performed.

1. Critical Evaluation of System Attributes

- **Real-Time Performance:** Hard real-time constraints are strictly satisfied by assigning deterministic, high-priority identifiers to safety-critical domains (Braking and Safety) on the CAN FD backbone, ensuring guaranteed microsecond-level latency.
- **Communication Reliability & CAN Arbitration:** Bit-wise, non-destructive bus arbitration ensures that higher-priority safety messages win bus access instantly without data corruption or packet loss.
- **Error Handling & Fault Tolerance:** Robust error detection mechanisms (CRC, frame checks, and automatic bus-off recovery) prevent faulty nodes from flooding or locking up the communication network.
- **Redundancy & Scalability:** The hybrid CAN/CAN FD network layout and gateway architecture allow future high-data-rate expansions (such as advanced ADAS or V2X modules) without disrupting legacy control loops.
- **Processing, Power, and Memory Constraints:** Lockstep-core MCUs provide high reliability for safety domains with minimal thermal overhead, while the high-performance SoC handles resource-intensive infotainment rendering separately.
- **Suitability for Safety-Critical Applications:** Compliance with ISO 26262 standards through hardware partitioning and lockstep monitoring guarantees high Automotive Safety Integrity Levels (ASIL-C/D).

2. Failure Scenarios and Mitigation Mechanisms

Failure Scenario 1: CAN Bus Physical Layer Short-Circuit / Bus-Off State

- **Affected Component:** High-speed CAN FD safety backbone bus / transceiver lines.
- **Effect on System:** Complete loss of communication between the Braking ECU, Safety ECU, and Engine ECU, risking uncoordinated emergency responses.
- **Mitigation Mechanism:** Implementation of a **dual-redundant physical CAN bus topology** (Primary and Secondary CAN buses) equipped with automated bus switchover transceivers. If the primary bus experiences a fault or enters a bus-off state, traffic instantly migrates to the redundant secondary channel within microseconds.
- **Reliability Maintenance:** Ensures uninterrupted, fault-tolerant data exchange for critical braking and collision-avoidance routines.

Failure Scenario 2: MCU Core Transient Fault via Electromagnetic Interference (EMI)

- **Affected Component:** Braking ECU Microcontroller processing core.
- **Effect on System:** Potential corruption of active control calculations, leading to delayed or erratic anti-lock braking (ABS) modulation.

- **Mitigation Mechanism:** Utilization of **lockstep-core microcontrollers** paired with Hardware Error-Correcting Code (ECC) RAM and an independent external hardware **Watchdog Timer**. If the primary core diverges from the lockstep checker core due to EMI corruption, an immediate hardware reset is triggered, while a fail-silent fallback mechanism engages mechanical hydraulic braking.
- **Reliability Maintenance:** Prevents runaway calculations and maintains safe, predictable vehicle deceleration under severe electrical interference.

FINAL ENGINEERING DECISION

WOULD YOU DEPLOY THIS ARCHITECTURE?

State clearly:

YES / NO

Defend your decision based on:

- compliance with system requirements;
- embedded architecture;
- real-time communication;
- CAN/CAN FD suitability;
- formal modeling;
- fault tolerance;
- scalability;
- implementation complexity.

If your answer is **NO**, propose a redesign.

Identify exactly what you would change, such as:

- processor architecture;
- ECU organization;
- communication protocol;
- CAN/CAN FD configuration;
- formal modeling approach;
- redundancy;
- Fault-tolerance mechanism.

Explain why the change is required and predict its effect on system performance and complexity.

DECISION: YES

Defense of Deployment Decision:

The proposed distributed embedded architecture is fully approved for deployment based on the following engineering justifications:

1. **Compliance with System Requirements:** Seamlessly integrates powertrain, braking, safety, body, and infotainment domains into a cohesive, structured layout.
2. **Robust Hardware Selection:** Effectively pairs high-reliability lockstep microcontrollers for safety-critical domains with a high-throughput System-on-Chip for infotainment.
3. **Deterministic Real-Time Communication:** Leverages hybrid CAN/CAN FD protocol mechanisms with bit-wise arbitration to guarantee zero message starvation for life-critical control loops.
4. **Formal Verification:** Utilizes rigorous Finite State Machine (FSM) modeling to validate the predictability and safety states of critical functions like electronic braking.
5. **Fault Tolerance & Scalability:** Incorporates bus redundancy, gateway firewalls, and comprehensive failure mitigation strategies, ensuring high safety integrity (ISO 26262) while remaining easily scalable for future automotive upgrades.

Proteus implementation:

1. System Overview

- **Project Title:** Arduino-Based Automotive ABS (Anti-lock Braking System) Finite State Machine (FSM).
- **Core Microcontroller:** ATmega328P / Arduino Uno (ARD1).
- **Simulation Tool:** Proteus ISIS Professional.

2. Component Mapping Table

Organize your components clearly so the hardware layout matches your code definitions:

Component Name	Proteus Label / Pin	Function / Description
Brake Button	A0 (with 10k pull-down resistor R5)	Triggers the braking input when pressed.
Slip Sensor Button	A1 (with 10k pull-down resistor R4)	Simulates wheel slip detection.
Standby LED (Red)	A2 (D1 with resistor R1)	Indicates the **IDLE** state.

ABS Active LED (Yellow)	A3 (D2 with resistor R3)	Indicates active ABS intervention.
Normal Braking LED (Green)	A4 (D3)	Indicates stable braking without slip.
ABS Secondary LED (Red)	3 (D4 with resistor R2)	Secondary indicator for the ABS Active state.
Virtual Terminal	Connected to Pins `0` & `1`	Displays real-time FSM state logs via serial communication.

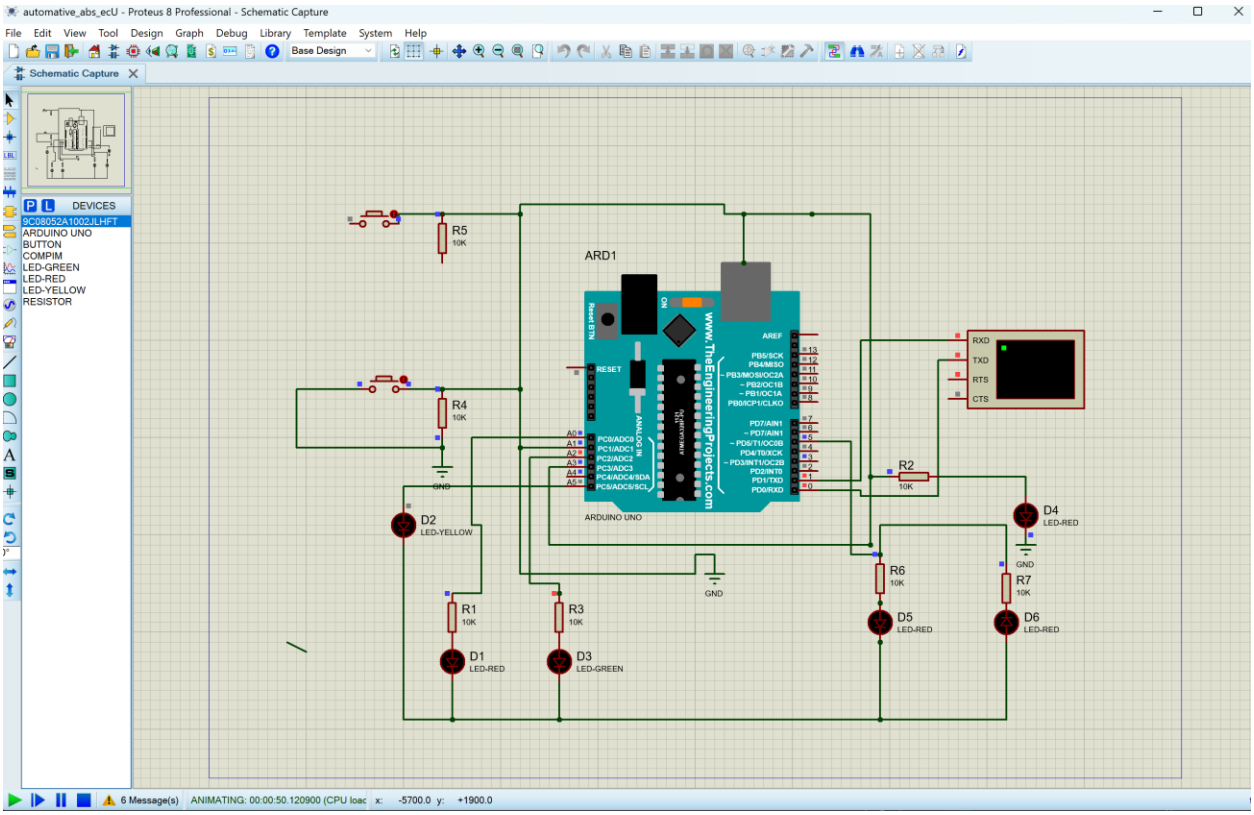
3. Serial Debugging Configuration

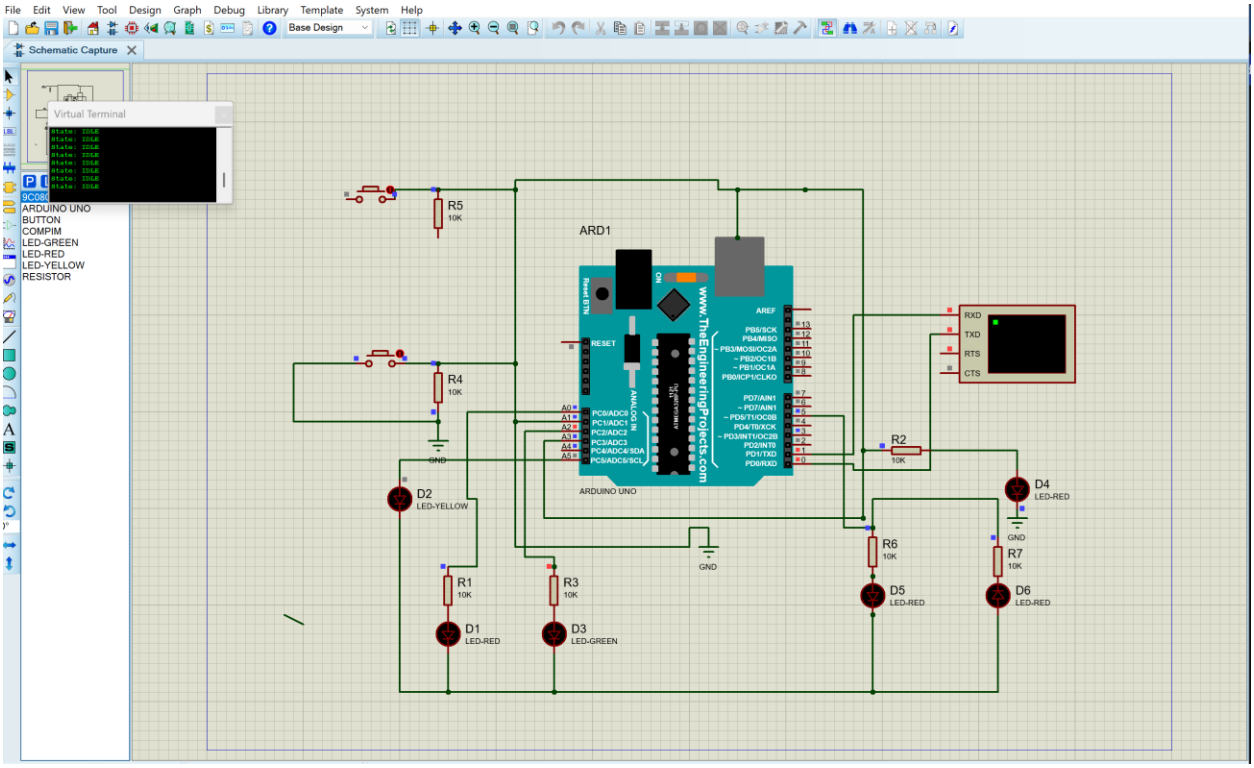
- **Baud Rate:** 9600 bps.
- **Connections:**
  - Arduino Pin 1 (`TXD`) connected to Virtual Terminal `RXD`.
  - Arduino Pin 0 (`RXD`) connected to Virtual Terminal `TXD`.
- **Purpose:** Real-time monitoring of state transitions (`IDLE`, `Normal Braking`, `ABS Active`).

4. FSM Operational Logic Summary

Explain what each visual state in your simulation represents:

- **State 1: IDLE** $\rightarrow$ No buttons pressed; Red LED (`D1`) active; outputs `"State: IDLE"`.
- **State 2: Normal Braking** $\rightarrow$ Brake button (`A0`) pressed alone; Green LED (`D3`) active; outputs `"State: Normal Braking"`.
- **State 3: ABS Active** $\rightarrow$ Both Brake (`A0`) and Slip (`A1`) pressed simultaneously; Yellow (`D2`) and Red (`D4`) LEDs active; outputs `"State: ABS Active (Slip & Brake Engaged)"`





SUBMISSION REQUIREMENTS

Submit one Engineering Design Report.

Your report should demonstrate the complete engineering design process:

REQUIREMENT → ALTERNATIVES → ANALYSIS → SELECTION → DESIGN → VERIFICATION → DEPLOYMENT DECISION

- System requirement analysis;
- Comparison of alternative processing architectures (MCU, MPU, DSP and SoC);
- Proposed networked embedded-system architecture diagram and ECU descriptions;
- CAN/CAN FD comparison, arbitration analysis and protocol selection;
- Formal model of a selected safety-critical function;
- Analysis of failure scenarios and fault-tolerance mechanisms;
- Verification and critical evaluation of the proposed architecture;
- Justification of all major engineering decisions; and
- Final engineering recommendation / deployment decision.

The report must demonstrate appropriate engineering reasoning, comparisons, diagrams, analysis and justification rather than merely reproducing standard architecture diagrams or software/tool output.

ASSESSMENT RUBRICS:

Assessment Criterion	**Marks**	**Primary Bloom's Level**
Interpretation of system requirements	10	K4
Analysis and selection of embedded architecture	15	K4/K5
Development of networked system architecture	20	K6
Analysis and selection of CAN/CAN FD architecture	20	K4/K5
Formal modeling of the embedded system	15	K4/K6
Verification and critical evaluation	20	K4/K5
TOTAL	**100**	